

Change Report

Cohort 3

Group 11

Team Aubergine

Joshua Wainwright, Piotr Koziol, Sarvesh Sridhar, Harrison
Barrans, Daniel Thwaites, Callum Newton, Arnav Jamidar,
Harry Turner

INTRODUCTION

When changing the assessment one deliverables we used various processes, tools and conventions to ensure up to date information was added and changes were made note of. First we'd plan to make changes via either group discussion or through google docs collaboration and then further utilised google docs to produce the new deliverables. Whilst producing the new deliverables we made sure to use the same referencing system so to prevent any inconsistencies from appearing. For code related changes we'd make note of them by using github to change the code as a team whilst using Github issues to document any notable problems and changes. After we'd made the changes we'd review them using group discussion to see if anything had changed since the change whilst using the google suite to directly compare the new version with the old version.

New and old documentation can be found on the website here:
<https://uoy-eng1-3-11.github.io/website-2/>

i. Req1:

The Introduction for the requirements deliverable provided a clear and thorough overview of the project requirements for the game. It explains the purpose and scope of the document, describes how user requirements were elicited from client discussions, and shows how system and constraint requirements were derived from these user needs. It also references relevant standards (IEEE 29148-2011 for requirements engineering and IEEE 830 for the 'Shall/Should/May' prioritisation framework), and clearly explains the approach to requirement traceability using Single Statements of Need (SSoNs). Since the section already establishes context, methodology, and traceability in a well-structured manner, we concluded that no updates were needed to be made.

Two additional requirements (UR-15 and UR-16) have been incorporated to enhance the game's functionality, meet the client's requirements and therefore better align with the assessment criteria for Assessment 2, these are as follows:

1. UR-15 : Leaderboard Feature

- Importantly, this requirement aligns with Assessment 2's requirement to demonstrate data collection, scoring, and comparison among users via leaderboards. The inclusion of a leaderboard displaying top players and their respective highest scores encourages competition and enhances user engagement. This feature also supports performance tracking, which is essential for player motivation and replay value.

2. UR-16 : Achievement System

- This requirement also supports Assessment 2's requirement of implementing achievement systems. Introducing an achievement system

based on the completion of specific in-game tasks promotes goal-driven interaction and increases user satisfaction. It enhances gameplay longevity and provides a sense of progression and reward.

These requirements were added to ensure the system meets both user expectations and the client's needs, while also improving interactivity, motivation, and overall user experience. Furthermore, these requirements link with existing functional requirements of the system being easily extensible to support new levels or difficulty modes (FR-13).

At the same time, we reviewed the Non-Functional Requirements (NFRs) and project constraints and found them to be well-defined. The non-functional requirements cover all key aspects of the project, including security, reliability, performance, maintainability, usability, accessibility, scalability, resilience, and documentation. Each requirement is linked to relevant user requirements and includes measurable criteria, making them practical to implement and test.

The project constraints covering design, process, and financial considerations are also still relevant and clear. They provide guidance on managing scope, time, and resources while prioritising core features like gameplay and accessibility. Since these sections already give clear direction and align with the client's requirements and assessment criteria, hence no updates were necessary.

The Functional Requirements (FRs) were also reviewed and found to be complete and aligned with user requirements. Each functional requirement effectively supports one or more user requirement(s), covering core system functionality such as game launch and cross-platform compatibility, menus, timers, feedback mechanisms, tutorials, UI consistency and extensibility for features. The FRs also address potential gameplay enhancements, like hidden events, website deliverables, including documentation and progress updates. Because the functional requirements already define the system's expected functionality in sufficient detail and are practical to implement and test, we decided that no changes were necessary.

ii. **Arch1:**

The main elements and components lists were edited to better reflect the present modifications and requirements. This resulted in the inclusion of:

- The Entity Manager (element):
 - Entities were handled manually in most cases, which was not scalable. To resolve this, the entity manager was listed to handle the logic, lifetimes, introductions, and timings of each entity present on the map.
- Score (manager):
 - Score is an integral part of the project, providing users with a motive to engage with the program even after a first playthrough. Score is required to facilitate other introductions to the program as well, such as the Achievements and Leaderboards sections.

- Achievements (element & manager):
 - New requirements are present for this deliverable, which the previous assignments code does not directly support. One area affected is the introduction of achievements, which have been newly listed in this section, present to facilitate user exploration and engagement.
- Leaderboards (element & manager):
 - As above, Leaderboards are another component handled in the new architecture. These systems are designed to record scores gathered throughout the course of gameplay on the condition that the score is greater than the previous 5 recorded values.

The flow charts present on the document were also updated. This is to give a more accurate view of the code/design to be implemented, as well as to facilitate new additions required for project submission. Figure 1 has been modified to show our heavier reliance on custom screen objects, as well as the introduction of score handling

Figure 4 represents the game logic used, which has also been heavily modified to fit criteria. To begin with, events and the effects on entities are accounted for, supporting the introduction of new items, interactables, and systems (e.g. score). End conditions were also evaluated to ensure our game followed the required timeline of events to provide a fair gameplay loop.

Figure 5 relates to the user's planned gameplay loop throughout the project. This diagram focuses heavily on the positions and interactions between the player, and items & entities. This is in place to facilitate the events listed in the Figure 4, as this will enable the game to feel more responsive to inputs. It has also placed heavy emphasis on the exploration aspect of the project, present as this is the main aspect of a "maze" game.

iii. **Risk1:**

When updating the risk assessment we removed the starting paragraphs aside from the last as they referred to the previous groups strategies, which were not our own. We made sure to add in descriptions of the risk types as it is not clear how the types work. We also removed the 'Technological' risk category/type and it was subsumed into the product risk category/type.

We have added two new risks (R14 and R15) to the project's risk register (see below). Two risks were removed from the previous risk assessment but a few were recategorised into the correct categories. These are R3, R11 and R12. We also added in more information onto the mitigation on R6 and R10 to more accurately reflect our groups strategies. Descriptions were also reworded on R1 and R10 to improve clarity

Two additional Risks (R14 and R15) additions are as follows:

1. R14: *Group low cohesion with communication between technical and non-technical team*
 - a. Risk R14 relates to our team structure. Since the team is divided into technical and non-technical subgroups, there's a chance for communication gaps or misalignment of tasks. Highlighting this risk allows us to put strategies in place—like clear communication channels and documented task assignments—to support better collaboration and keep the project running smoothly.
2. R15: *Breakdown in continuous Integration with new project*
 - a. Risk R15 relates to the integration of the new project from the previous group. If we do not relate back to their documentation and understand it, it would be easy for our team to hit roadblocks where we do not understand what they've done.

iv. **Plan and method:**

Compared to the original group we took over we have used multiple different tools and removed some

- Asana -> Github issues
 - Our team had no prior experience with Asana as a tool and we felt it was unnecessary to learn a new software to when we had a tool we understood already when we already knew github issues which integrated with Github which we already were using making making Asana obsolete
- Change: Discord -> Whatsapp
 - We added in whatsapp as a tool we used for group communication and removed discord. In part one of the project we used whatsapp worked excellently so we decided to use it as a tool as its effectiveness had already been proven. Discord was overkill for our needs as it provided no extra functionality as a tool in our use case compared to whatsapp
- Addition: JUnit
 - For the second part of the project, we needed to do automated software testing to ensure proper functionality of our code. JUnit was chosen as it has good compatibility with LIBGDX and is a standard tool in the java ecosystem.
- Addition: JaCoCo
 - We used JaCoCo as it provides high quality code coverage reports for the testing which are accessible for all team members without needing a development environment to be made reducing complexity.
- Addition: Text Editor / IDE
 - We added this in as we used Text Editor / IDEs to edit and maintain the code. The tools we used are non-specific as all of our code is in standard java. We added in the main IDEs we used however.
- Removal: Scrum

- Our process did not require the use of Scrum as it was unnecessary to learn and understand the process as our current methods were more than enough to produce our project deliverables.

We changed the evolution of our plan to show how our plan changed throughout the second part of the project instead of the first. Additionally adding in a new task table to replace the previous groups. Improving on the layout to improve readability.